Shri Govindram Seksaria Institute of Technology and Science, Indore

A report Submitted In Partial Fulfilment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Department of Computer Science and Engineering

Government Scheme Portal

Guided By:
Prof. Priyanka Bamne
Ms. Meghna Chandel

Submitted By:
Anushtha Singh Kushwah – 0801CS233D01
Harshita Bamniya – 0801CS233D04
Mohan Manjhi – 0801CS233D05
Shivani Choudhary – 0801CS33D08
Shreyash Tiwari – 0801CS233D09

Shri Govindram Seksaria Institute of Technology and Science, Indore

RECOMMENDATION

This project report, titled **Government Scheme Portal**, is being developed by **Anushtha Singh Kushwah, Shreyash Tiwari, Mohan Manjhi, Harshita Bamniya, and Shivani Choudhary** under the guidance of **Prof. Priyanka Bamne** and **Ms. Meghna Chandel** in the Department of Computer Science and Engineering, **S.G.S.I.T.S. Indore**.

The project is being undertaken as part of the academic curriculum for the B.Tech program and is currently in progress during the 6th semester. It aims to develop a web-based platform to help users find relevant government schemes efficiently through search functionality, chatbot integration, and multilingual support.

This document serves as a record of the work completed so far and outlines the project's objectives, methodology, and expected outcomes.

Prof. Priyanka Bamne
(Guide)

Ms. Meghna Chandel
(Co-Guide)

Shri Govindram Seksaria Institute of Technology and Science, Indore

CERTIFICATE

This is to certify that the project report, titled **"Government Scheme Portal"**, is being developed by **Anushtha Singh Kushwah, Shreyash Tiwari, Mohan Manjhi, Harshita Bamniya, and Shivani Choudhary** under the guidance of **Prof. Priyanka Bamne** and **Ms. Meghna Chandel** in the Department of Computer Science and Engineering, **S.G.S.I.T.S. Indore** during the academic year 2025, is a record of their own work and is accepted in the partial fulfillment for the award of the degree of Bachelor of Computer Science Engineering.

Internal Examiner
Date:

External Examiner
Date:

Contents

Abstract

In recent years, the Indian government has launched numerous welfare schemes targeting various sectors such as education, agriculture, healthcare, employment, and social justice. However, due to fragmented information across multiple government websites, accessing and understanding these schemes remains a challenge for the common citizen.

The Government Scheme Portal is a centralized platform designed to address this gap. It aims to empower users—particularly those with limited digital literacy—by simplifying access to relevant scheme information through automated web scraping, intelligent search, and language support. The platform consolidates data from official government portals, processes it into a user-friendly format, and offers interactive chatbot-based assistance for queries.

By integrating speech-to-text, regional language support, and a responsive web interface, the portal enhances accessibility for rural populations, elderly citizens, and those with disabilities. The implementation of machine learning algorithms enables personalized scheme recommendations based on user profiles and eligibility criteria, significantly improving the likelihood of citizens finding relevant assistance programs. Additionally, the platform's intuitive visual design and step-by-step application guidance removes traditional barriers to welfare program enrollment.

Future development includes a mobile application to increase reach and usability across devices, offline functionality for areas with limited connectivity, and integration with digital identity verification to streamline application processes. This project represents a significant step toward improving digital governance and ensuring that the benefits of public welfare schemes are accessible to every eligible citizen regardless of their technical proficiency, language preference, or geographic location.

Chapter 1
Introduction

1.1 Background and Context

In India, numerous government schemes are launched to benefit different sections of society. However, this information is scattered across multiple government websites, making it difficult for citizens to find relevant schemes. Many citizens, especially those from rural areas and elderly populations, miss out on benefits they are eligible for simply due to information barriers.

1.2 Motivation for the Project

- Many government websites are not user-friendly, especially for older citizens
- Lack of a centralized platform to search and filter schemes
- Existing platforms do not support regional languages, making it difficult for non-English speakers to access information
- Digital divide creates barriers for those with limited technical skills
- Underutilization of benefits due to information inaccessibility
- Growing smartphone penetration creates opportunity for mobile-friendly solutions

1.3 Objectives and Scope

The main goal is to create a user-friendly platform that allows users to:

- Search for government schemes based on keywords
- Get personalized recommendations based on eligibility
- Use a chatbot to find relevant schemes through conversation
- Convert text into different local languages for better accessibility
- Enable speech-based queries for users who are not comfortable typing
- Provide clear eligibility criteria and application processes for each scheme

Chapter 2
Literature Review

2.1 Related Work

Our examination of existing solutions reveals several approaches to providing government scheme information:

1. Government Portals:
    - MyGov Portal: Focuses on citizen engagement but offers limited scheme discovery features
    - National Portal of India: Provides information across departments but lacks user-centric organization
    - PM Kisan Portal: Well-designed for a specific scheme but represents the fragmented approach
    - State Government Websites: Inconsistent design and information architecture across states
2. Private Platforms:
    - IndiaFilings: Provides scheme information with business services but limited to major schemes
    - GovInfo by Factly: Consolidates information in journalistic format but lacks interactivity
3. Mobile Applications:
    - UMANG: Combines multiple services but prioritizes delivery over discovery

2.2 Gaps in Existing Knowledge

Based on our review, we identified several critical gaps:

- No comprehensive aggregation of schemes across government levels
- Limited search intelligence and natural language understanding
- Absence of personalization based on user profiles
- Insufficient accessibility features for diverse user needs
- Lack of conversational interfaces for guided discovery
- Outdated information across many platforms
- Complex navigation structures that reflect government organization rather than user needs

Chapter 3
Problem Statement

The lack of a centralized, user-friendly, and accessible platform for discovering and understanding government schemes creates substantial barriers for Indian citizens attempting to access welfare benefits they are entitled to. This problem is particularly acute for digitally less confident users, speakers of regional languages, elderly citizens, and rural populations. The current ecosystem of fragmented portals with complex interfaces, limited search capabilities, and inconsistent language support results in eligible citizens missing opportunities for critical support in areas such as education, healthcare, financial assistance, and social security.

The Government Scheme Portal project addresses this problem by developing a unified, accessible platform that consolidates scheme information, implements intelligent search and conversational interfaces, supports multiple Indian languages, and provides personalized recommendations, thus democratizing access to welfare scheme information across demographic and technological divides.

Chapter 4
Methodology

4.1 Detailed Plan of Action

Our methodology combines user-centered design with agile development principles:

1. User Research and Design:
    - Conduct user interviews with diverse demographic groups
    - Create user personas representing different citizen profiles
    - Develop journey maps and wireframes for testing
    - Iterate designs based on usability feedback
2. Data Collection and Structuring:
    - Identify authoritative sources for scheme information
    - Define standardized schema for scheme data
    - Implement web scraping for government websites
    - Develop data validation protocols
3. Technical Implementation:
    - Component-based architecture for maintainability
    - Responsive design for cross-device compatibility
    - RESTful APIs for data access
    - Modular services for core functionality
4. Natural Language Processing:
    - Train models for understanding scheme queries
    - Implement intent recognition for chatbot
    - Develop entity extraction for user information
    - Create dialogue management for conversations

4.2 Web Scraping Implementation

4.2.1 Overview of Web Scraping Process

We implemented a comprehensive web scraping pipeline to aggregate government scheme information from multiple official sources. Our parser-based approach uses Python libraries to extract structured data from government websites while respecting ethical web scraping practices.

4.2.2 Web Scraping Stack

- **Python Libraries:** Requests for HTTP requests, BeautifulSoup for HTML parsing
- **Data Processing:** Pandas for data manipulation and storage
- **Ethical Considerations:** Implementation of delays and proper headers

4.2.3 Parser-Based Scraping Workflow

Our parser-based scraping workflow follows these steps:

1. **Source Identification:** We identify and catalog relevant government websites containing scheme information
2. **URL Collection:** We extract URLs for individual scheme pages from index/listing pages
3. **HTML Fetching:** Using Python's requests library, we fetch the raw HTML content
4. **DOM Parsing:** BeautifulSoup parses the HTML structure into navigable DOM objects
5. **Data Extraction:** We extract specific elements like titles, descriptions, eligibility criteria, and benefits
6. **Data Cleaning:** We clean and normalize the extracted text data
7. **Structured Storage:** The cleaned data is stored in a structured format in our database

4.2.4 URL Extraction Process

The first step in our data collection pipeline was extracting URLs for individual scheme pages:

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

# Set headers to mimic browser behavior
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit
   /537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36',
    'Accept-Language': 'en-US,en;q=0.9',
    'Accept-Encoding': 'gzip, deflate, br',
    'Connection': 'keep-alive'
}

# List to store all scheme URLs
full_urls = []

# Main listing page URL
main_url = "https://example.gov.in/schemes"

# Get HTML content
response = requests.get(main_url, headers=headers)
soup = BeautifulSoup(response.content, 'html.parser')

# Extract all scheme links
scheme_links = soup.find_all('a', class_='scheme-link')
for link in scheme_links:
```

```
    url = link['href']
    full_urls.append(url)

# Save URLs to CSV
url_df = pd.DataFrame(full_urls, columns=['url'])
url_df.to_csv('scheme_urls.csv', index=False)
```

Listing 4.1: URL Extraction Code

4.2.5 Data Extraction from Individual Schemes

For each extracted URL, we implemented a process to visit the page and extract specific scheme details:

```
import pandas as pd
import requests
from bs4 import BeautifulSoup
import time

# Read URLs from previous step
urls_df = pd.read_csv('scheme_urls.csv')
scheme_urls = urls_df['url'].tolist()

# List to store all scheme data
all_schemes = []

# Process each URL
for url in scheme_urls:
    # Add delay to respect server resources
    time.sleep(1)

    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.content, 'html.parser')

    # Extract scheme details using selectors
    try:
        scheme_title = soup.find('h1', class_='scheme-title').text.strip()
        scheme_desc = soup.find('div', class_='scheme-description').text.
    strip()
        scheme_eligibility = soup.find('div', class_='eligibility').text.
    strip()

        # Create data dictionary
        data = {
            'Title': scheme_title,
            'Description': scheme_desc,
            'Eligibility': scheme_eligibility,
            'URL': url
        }

        all_schemes.append(data)
    except Exception as e:
        print(f"Error processing {url}: {e}")

# Convert to DataFrame and save
schemes_df = pd.DataFrame(all_schemes)
schemes_df.to_csv('schemes_data.csv', index=False)
```

Listing 4.2: Scheme Data Extraction Code

4.2.6 Ethical Web Scraping Practices

We implemented several measures to ensure ethical web scraping:

- **Request Delays:** We included a 1-second delay between requests using `time.sleep(1)` to avoid overwhelming servers and simulate human browsing behavior.
- **Browser Headers:** We used proper HTTP headers to identify our requests transparently.
- **Error Handling:** We implemented robust error handling to manage unexpected HTML structures or server responses.
- **Rate Limiting:** Our scraper was designed to respect the implicit rate limits of government websites.

4.3 Chatbot Implementation

4.3.1 Chatbot Architecture

We developed a conversational interface to help users discover relevant government schemes through natural language queries:

- **Frontend:** HTML, CSS, and JavaScript to create a responsive chat interface
- **Backend:** n8n webhook endpoint for processing queries
- **Language Support:** Hindi and English queries supported

4.3.2 Chatbot Features

- **Chat History:** Local storage for persisting conversation history across sessions
- **Typing Indicator:** Visual feedback showing when the bot is processing
- **Session Management:** Unique session IDs using UUID for tracking conversations
- **Error Handling:** Graceful responses when backend services are unavailable

4.3.3 Conversation Flow

1. User enters a query about government schemes (e.g., " ")
2. Query is sent to the n8n webhook endpoint
3. Backend processes the query using NLP techniques
4. Relevant schemes are identified and formatted as a response
5. Response is displayed to the user in the chat interface

4.4 Technology Stack

4.4.1 Frontend

- React.js for component-based UI development
- Tailwind CSS for responsive styling
- Accessibility components for inclusive design

4.4.2 Backend

- Node.js with Express.js for API development
- Search service for query processing
- Chatbot service for conversational interface
- Language service for translations

4.4.3 Database

- PostgreSQL via Supabase for structured data storage
- Analytics store for usage tracking
- Session management for user state

4.4.4 External Services

- OpenAI API for NLP capabilities
- Bhashini API for language translation
- Web scraping services for data collection

4.5 Project Timeline

- Phase 1: Requirements Gathering and Design (2 weeks)
- Phase 2: Database Development and Data Collection (3 weeks)
- Phase 3: Core Interface Implementation (4 weeks)
- Phase 4: Chatbot and Search Development (3 weeks)
- Phase 5: Language and Accessibility Features (2 weeks)
- Phase 6: Testing and Refinement (2 weeks)

4.6 UML Diagrams

4.6.1 Flow Chart

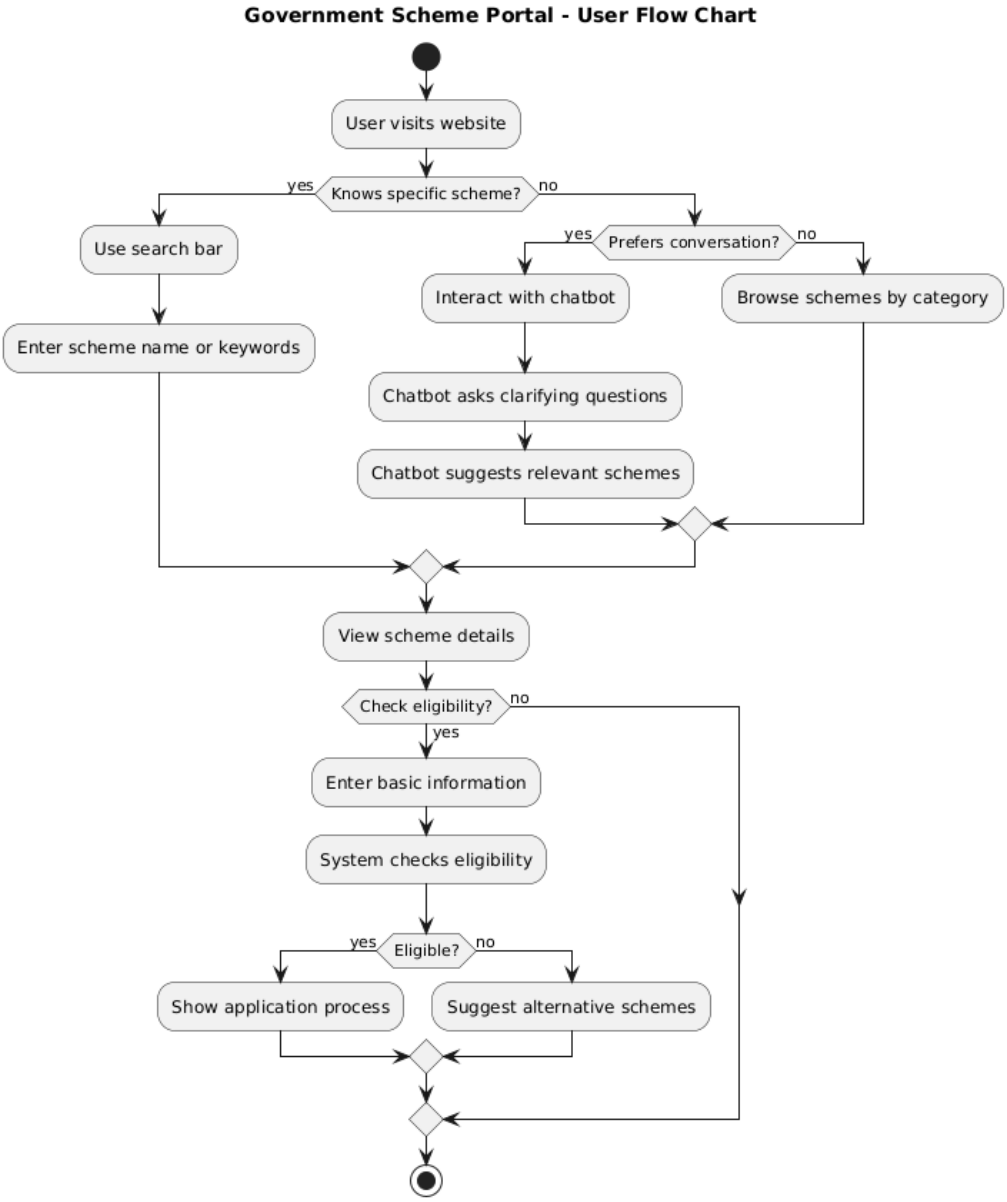

Figure 4.1: Flow Chart of Government Scheme Portal

4.6.2 Architecture Diagram

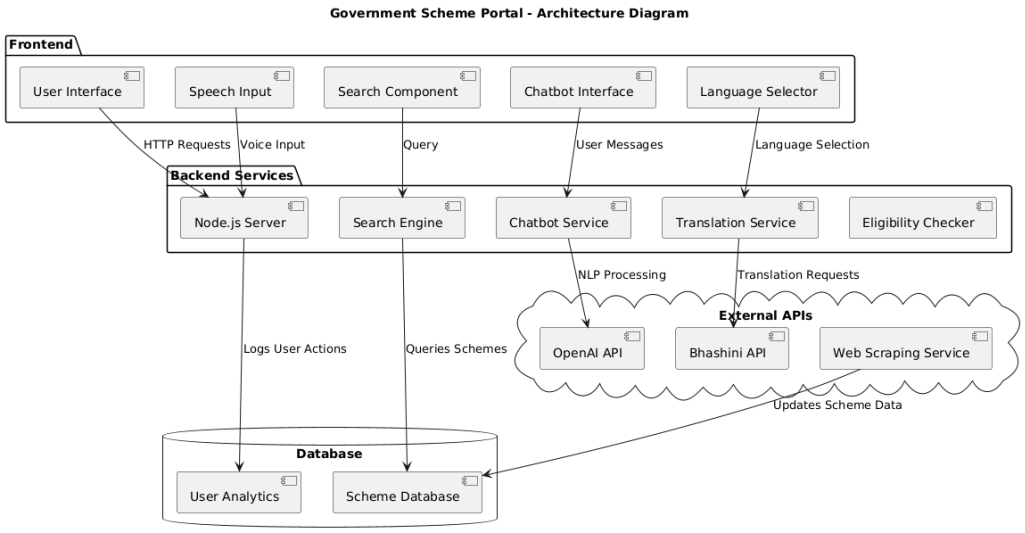

Figure 4.2: Architecture Diagram of Government Scheme Portal

4.6.3 Class Diagram

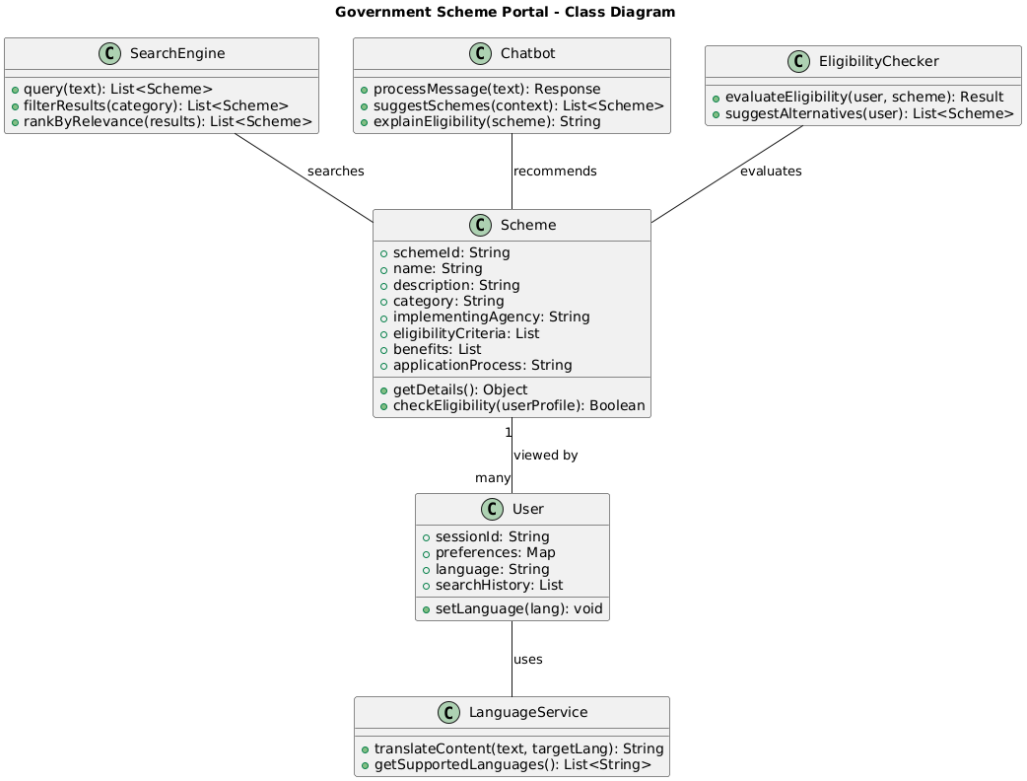

Figure 4.3: Class Diagram of Government Scheme Portal

4.6.4 Use Case Diagram

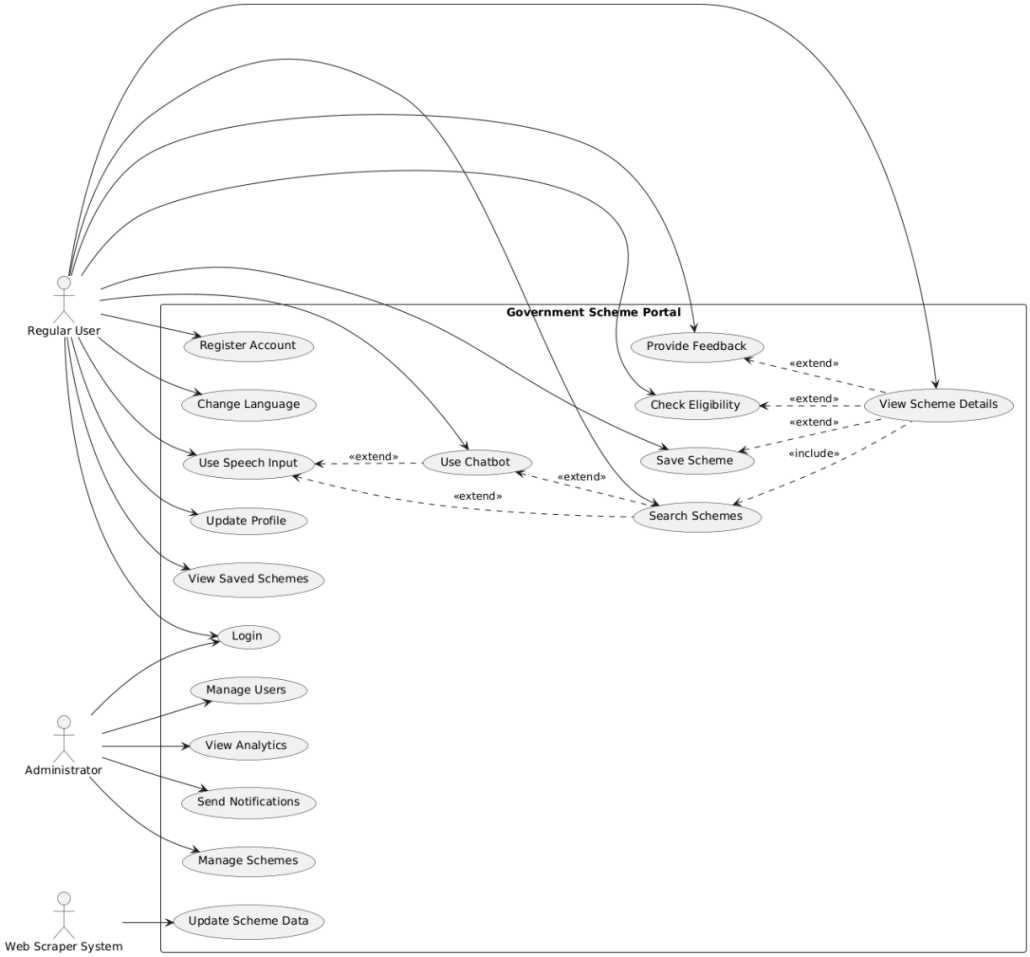

Figure 4.4: Use Case Diagram of Government Scheme Portal

4.6.5 Activity Diagram

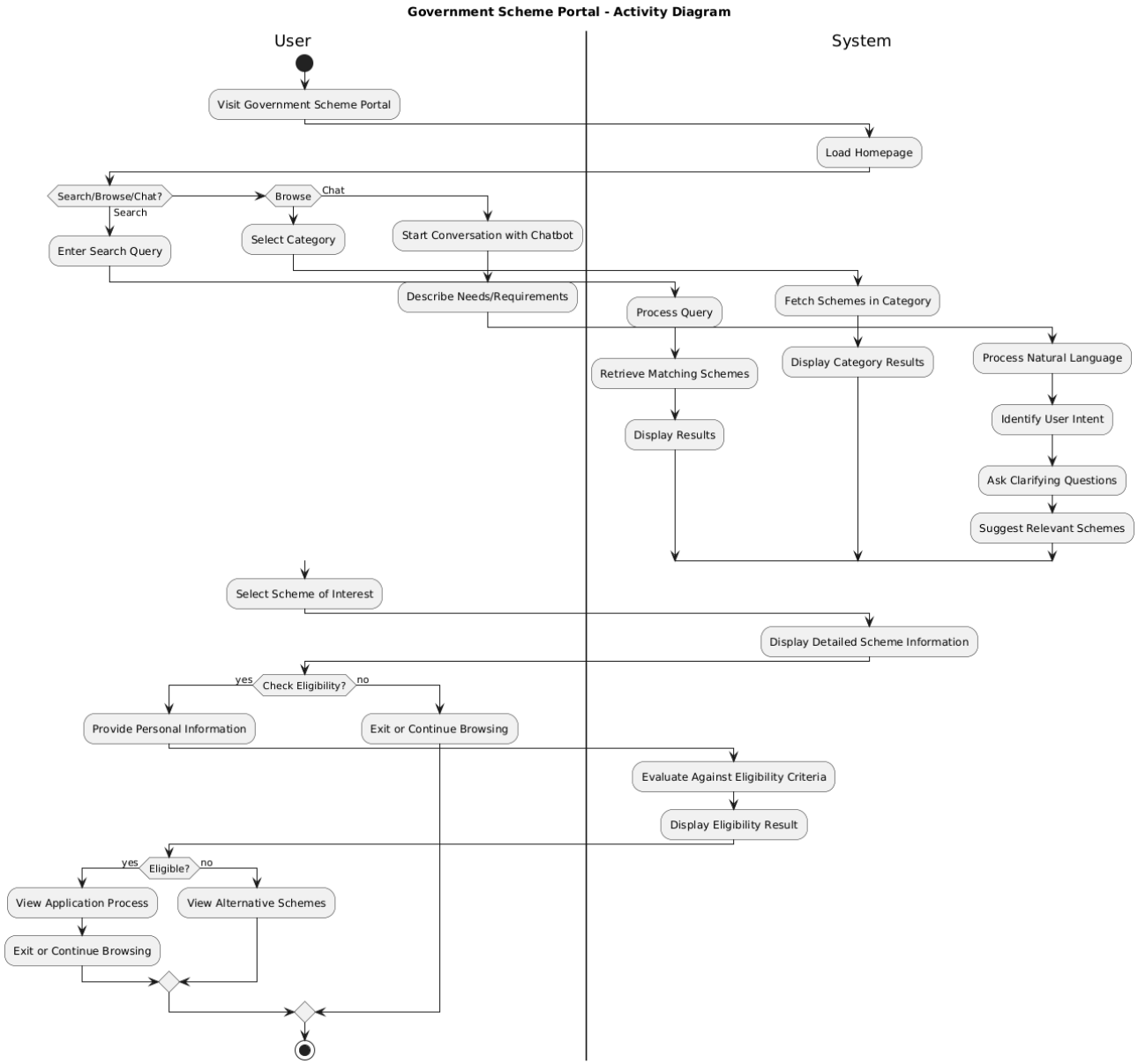

Figure 4.5: Activity Diagram of Government Scheme Portal

4.6.6 Data Flow Diagram for Web Scraping

Figure 4.6: Data Flow Diagram for Web Scraping Process

4.6.7 Website Prototype https://v0-voice-based-ui-prototype.vercel.app/

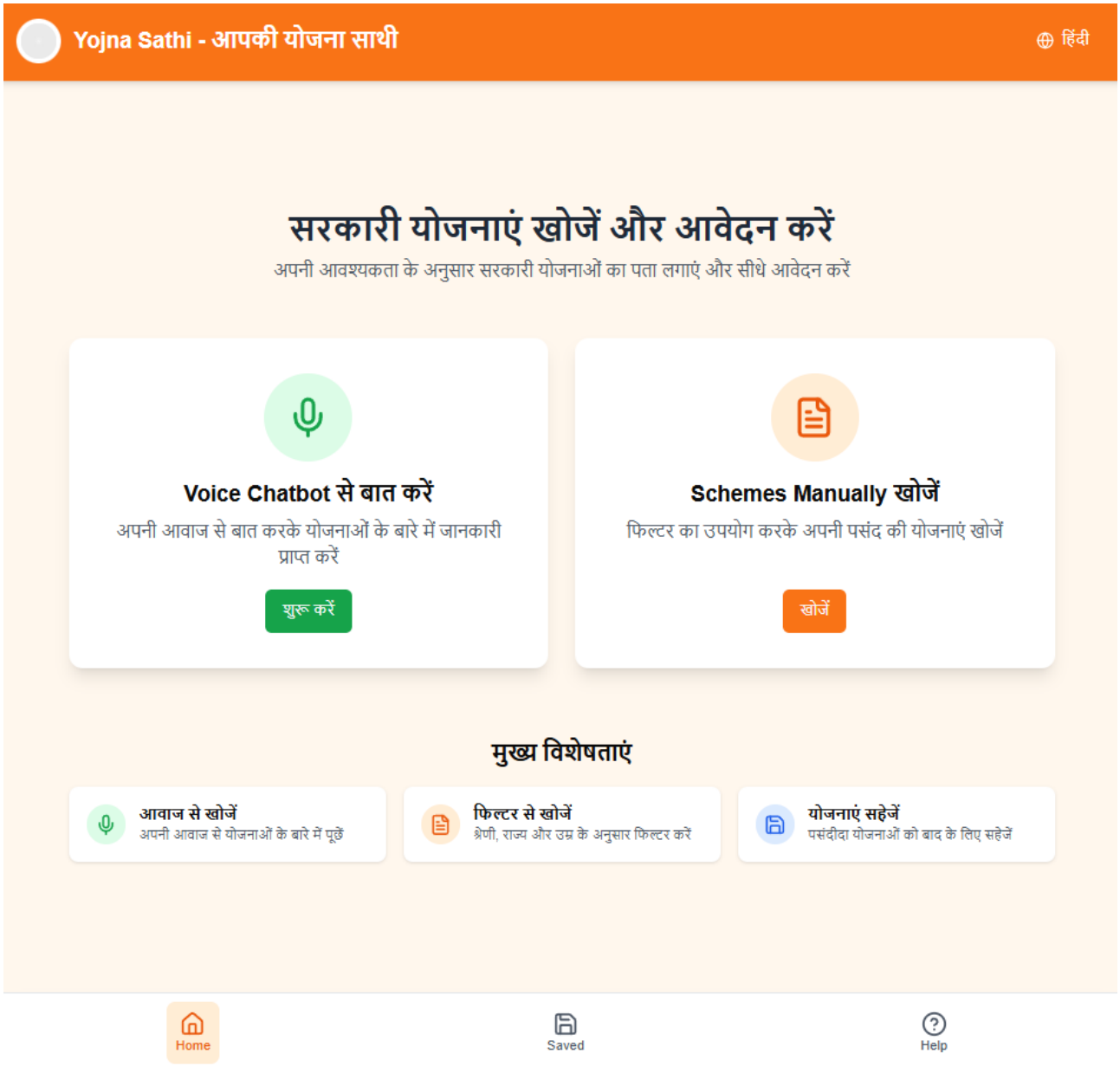

Figure 4.7: Government Scheme Portal Prototype Interface

4.6.8 Chatbot Interface

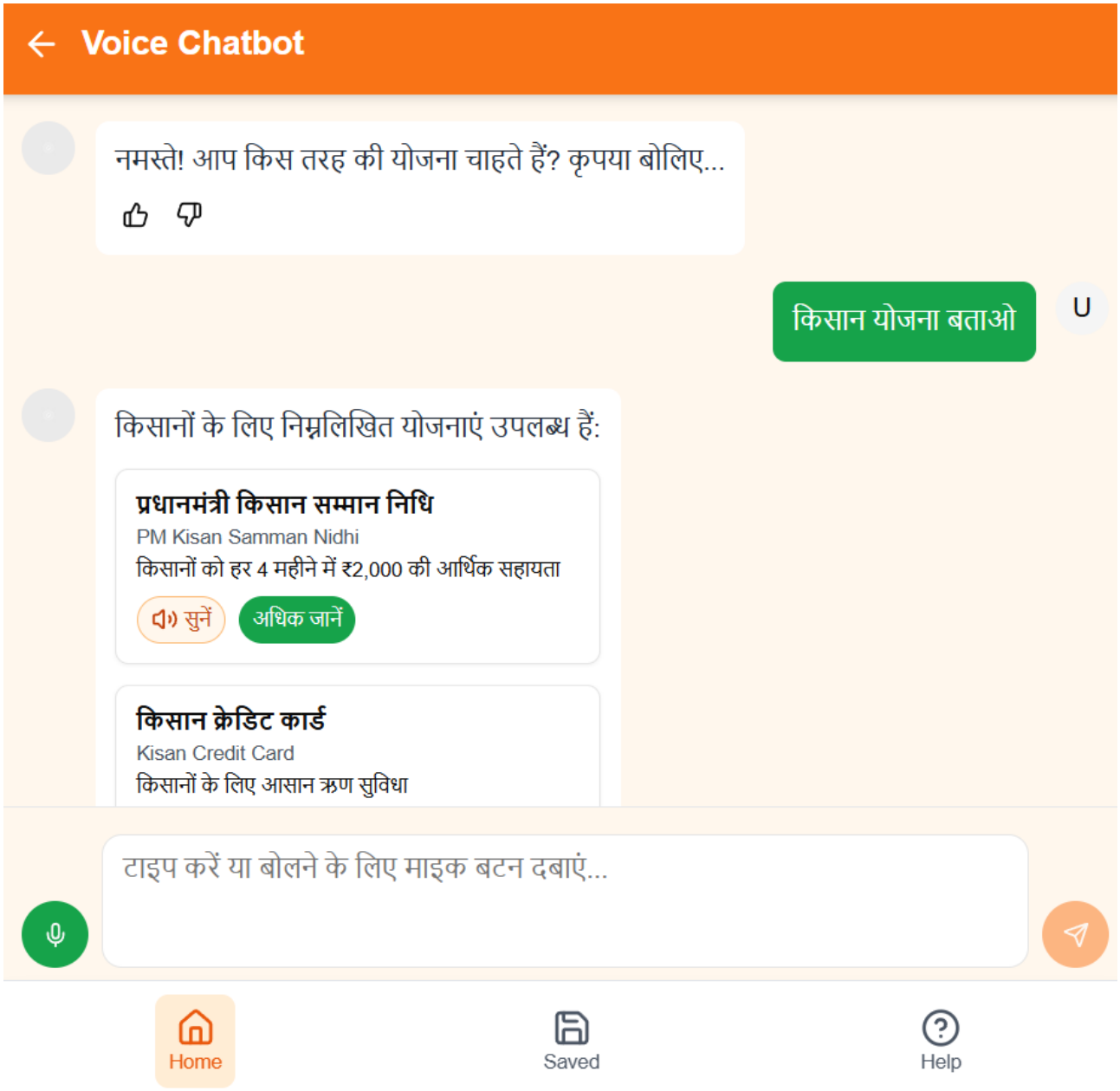

Figure 4.8: Government Scheme Portal Prototype Interface

Chapter 5
Expected Outcomes

The Government Scheme Portal is expected to deliver several key outcomes:

5.1 A Working Prototype with the following features

- Searchable database of government schemes compiled through automated web scraping
- Interactive chatbot with conversational abilities in Hindi and English
- Multi-language support for major Indian languages
- Speech-to-text functionality for voice queries
- Eligibility checker for scheme qualification

5.2 Comprehensive Dataset of Government Schemes

- Structured database of central and state government schemes
- Regular updates through automated web scraping pipelines
- Consistent format for scheme information across different sources
- Rich metadata including eligibility, benefits, and application procedures

5.3 Improved Accessibility

- Simplified information architecture for intuitive navigation
- Support for regional languages to overcome language barriers
- Voice interface for users with limited literacy or typing skills
- Mobile-responsive design for access across devices

5.4 Enhanced User Experience

- Personalized scheme recommendations based on user criteria
- Clear explanation of complex eligibility requirements
- Step-by-step guidance for application processes
- Consistent information structure across all schemes

5.5 Impact Metrics

- Increased discovery of relevant schemes by users
- Reduced time to find applicable government benefits
- Greater engagement from traditionally underserved demographics
- User satisfaction with information clarity and accessibility

Chapter 6
Resources Required

6.1 Software Requirements

For the implementation and execution of our project, the following software components are required:

- **Development Environment:** React, Node.js
- **Database Management System:** PostgreSQL
- **Version Control System:** Git/GitHub for source code management
- **API Integration Tools:** Tools for seamless communication with external services
- **Hosting and Deployment:** Vercel for efficient deployment and scalability
- **Web Scraping Tools:** Python with BeautifulSoup and Requests libraries

6.2 Hardware Requirements

To ensure a smooth development and testing process, the following hardware resources are necessary:

- **Development Machines:** High-performance computers or laptops for all team members
- **Testing Devices:** A range of devices, including mobile phones, tablets, and desktops, to ensure cross-platform compatibility
- **Server Infrastructure:** Cloud or on-premise servers for hosting and running backend services

6.3 APIs and Services

Integration with third-party APIs and services will enhance our application's functionality. The following APIs and services are required:

- **OpenAI API:** For advanced natural language processing capabilities
- **Bhashini API:** For multilingual translation support
- **Web Scraping Tools:** For automated data collection from various sources
- **Speech Recognition Services:** To enable voice-based interactions
- **n8n Webhook:** For chatbot backend processing

6.4 Development Tools

Efficient development and debugging require a suite of specialized tools. The following tools will be used:

- **Integrated Development Environment (IDE):** Visual Studio Code or an equivalent IDE
- **UI/UX Design Tools:** Figma for designing user interfaces and user experiences
- **API Testing Tools:** Postman for testing API endpoints
- **Browser Development Tools:** Built-in browser tools for debugging and performance analysis
- **Data Processing Tools:** Pandas for data manipulation and analysis

Each of these resources plays a crucial role in ensuring the efficiency, reliability, and scalability of our project. Proper allocation and management of these resources will facilitate the successful development and deployment of our system.

Chapter 7

References

- **myScheme Gov. Portal** Used for Webscrapping https://www.myscheme.gov.in/
- **Ministry of Electronics and Information Technology.** (2023). Digital India Programme. Government of India. https://digitalindia.gov.in/
- **Sharma, A., Patel, V., & Mehta, S.** (2023). User-Centered Design Approaches for E-Governance Platforms in Developing Nations. Journal of Public Administration and Digital Transformation, 12(3), 78-92.
- **Mehta, R., & Patel, J.** (2022). Conversational Interfaces for E-Governance: A Study of Adoption Patterns Among Rural Users. International Journal of Human-Computer Interaction, 38(5), 423-437.
- **National Informatics Centre.** (2024). Guidelines for Indian Government Websites. Ministry of Electronics and Information Technology. https://web.guidelines.gov.in/
- **World Bank.** (2023). Digital Government Readiness Assessment: India. World Bank Group. https://documents.worldbank.org/
- **MyGov Portal.** (2024). Government of India. https://mygov.in/
- **PM Kisan Portal.** (2024). Ministry of Agriculture & Farmers Welfare. https://pmkisan.gov.in/
- **UMANG App Documentation.** (2023). National e-Governance Division. Ministry of Electronics & Information Technology. https://web.umang.gov.in/
- **W3C Web Accessibility Initiative.** (2023). Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/
- **Richardson, L.** (2022). Beautiful Soup Documentation. https://www.crummy.com/software/BeautifulSoup/bs4/doc/
- **Reitz, K.** (2023). Requests: HTTP for Humans. https://requests.readthedocs.io/
- **McKinney, W.** (2022). pandas: Python Data Analysis Library. https://pandas.pydata.org/